

Dive into Python Library

실전 파이썬 라이브러리 활용

04 내 코드를 전세계 사람과 공유하기

목차 04 내 코드를 전세계 사람과 공유하기

1. 오픈 소스와 개발 생태계
2. PyPI에 내 패키지 등록하기
3. 학습 내용 정리 및 앞으로

1. 오픈 소스와 개발 생태계

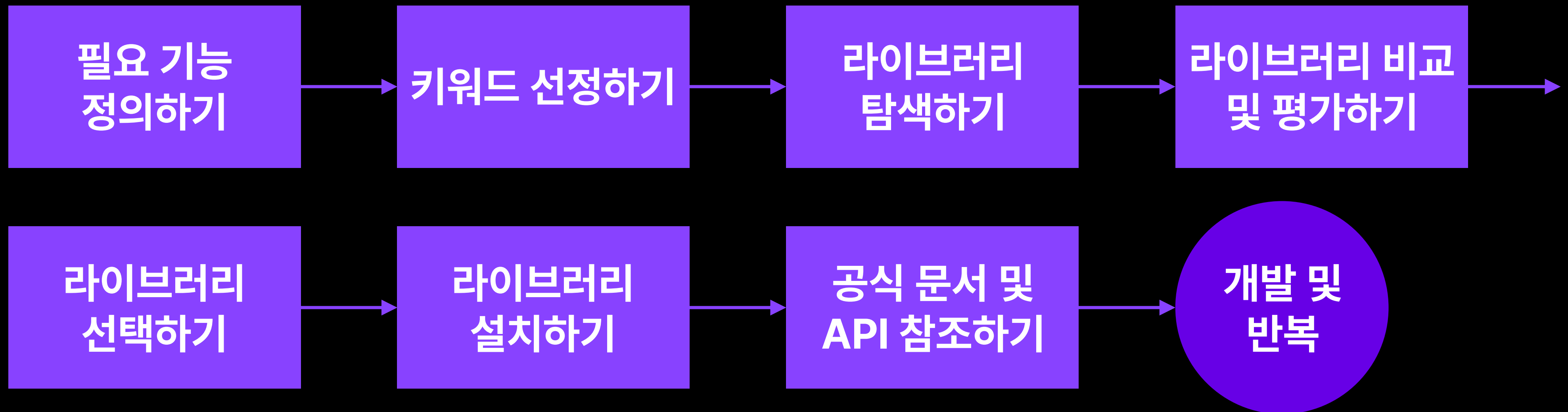


우리는 지금까지..

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

1. 오픈 소스와 개발 생태
계





Why?

이렇게 많은 사람들이 자신의 코드를 공유할까?

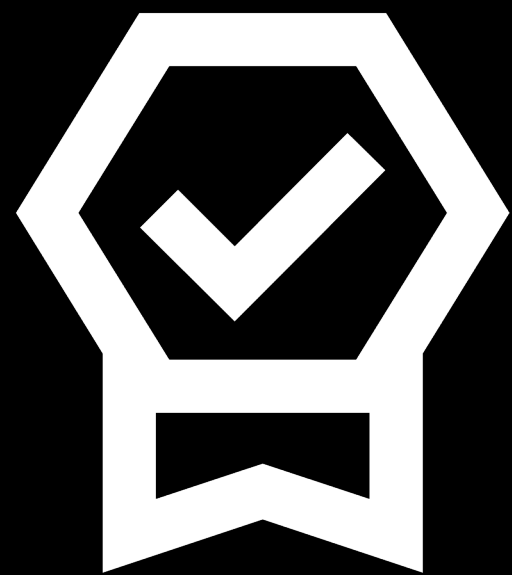


이유 1. 간접적인 이익

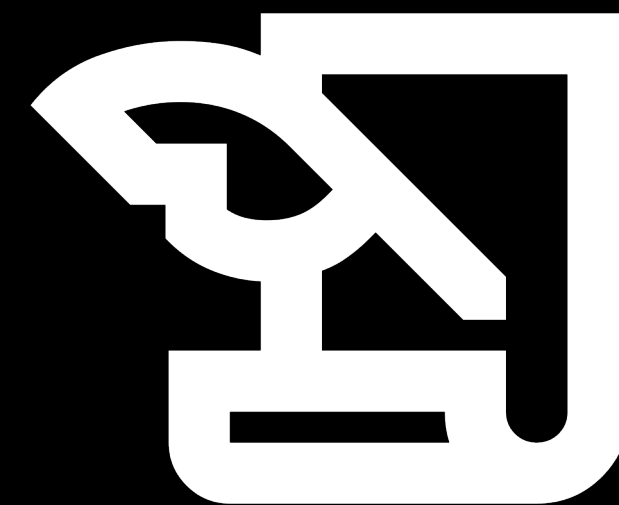
실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

1. 오픈 소스와 개발 생태
계



커뮤니티 기여와 명성



포트폴리오 구축

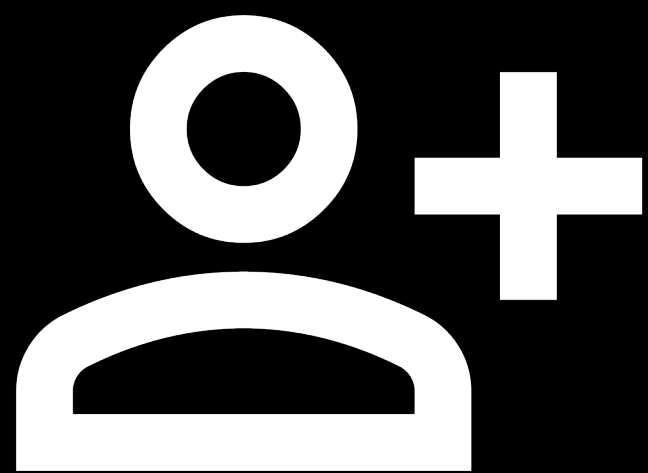


이유 2. 성장

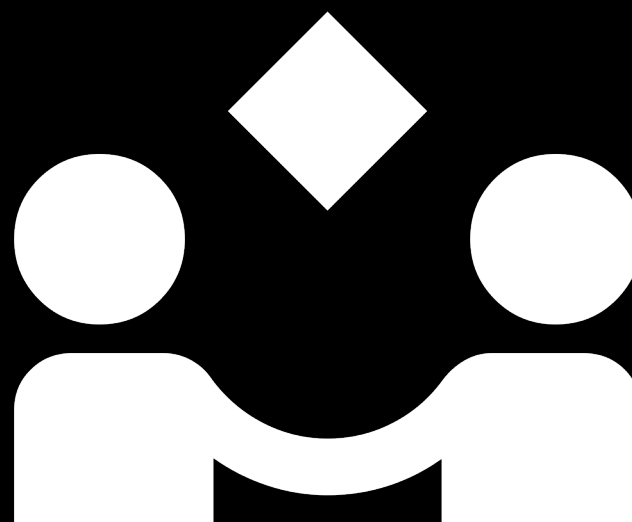
실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

1. 오픈 소스와 개발 생태
계



개인의 성장



피드백과 협업



이유 3. 지식 공유의 가치

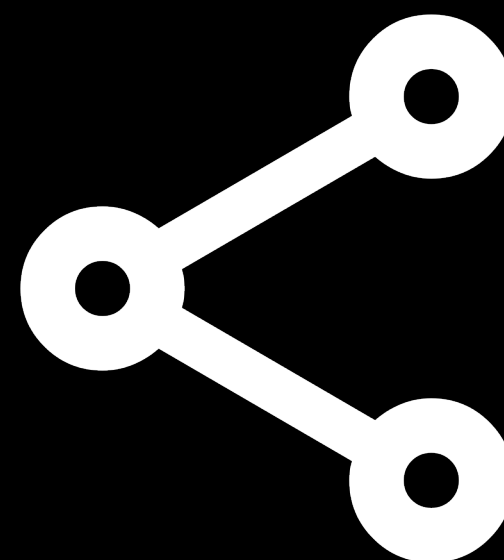
실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

1. 오픈 소스와 개발 생태
계



기술 발전 기여

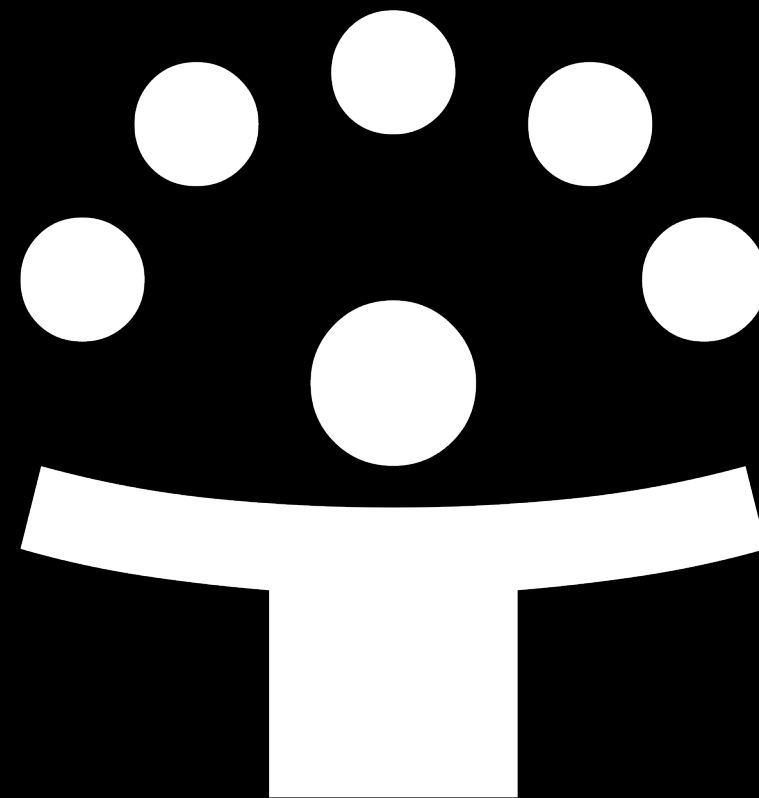


개방적인 소프트웨어
문화에 대한 지지



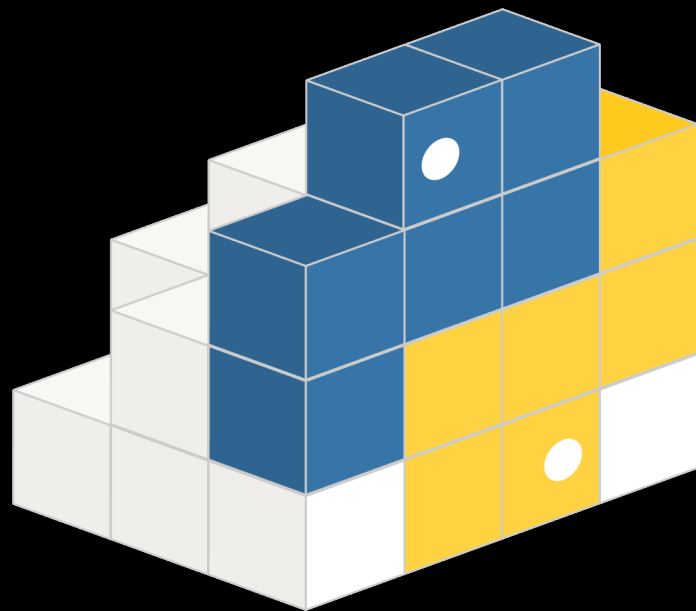
open source

- 소스 코드를 공개하여 누구나 자유롭게 사용, 수정, 배포할 수 있도록 허용하는 소프트웨어 개발 및 배포 방식
- 사용의 자유, 연구의 자유, 배포의 자유, 개선의 자유
- 리눅스, 파이썬, 깃 등이 대표적인 오픈 소스 프로젝트
- 기술의 발전에 있어 오픈 소스의 역할은 매우 중요
 - 소프트웨어 개발의 방식을 근본적으로 변화시킴
 - 혁신을 가속화하고, 기술의 대중화 촉진



내 코드를 전 세계적으로 공유하는 방법을 알아보자!

2. PyPI에 내 패키지 등록하기



PyPI

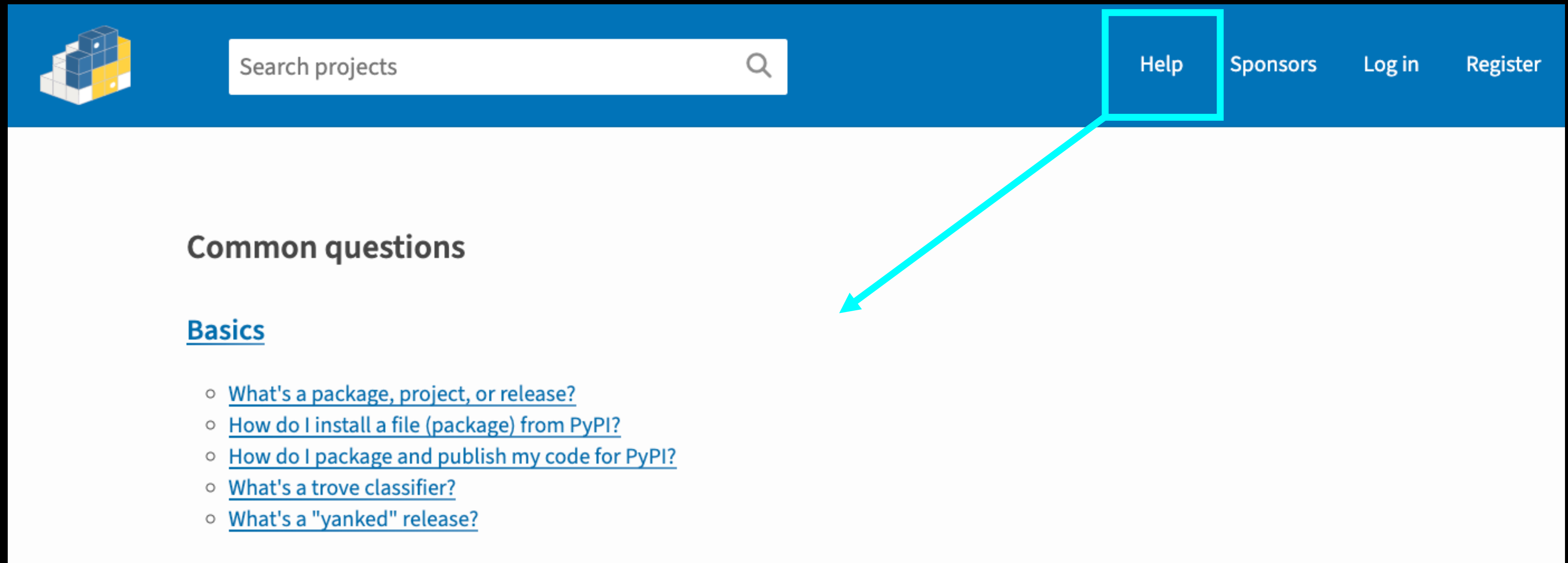
그들 중 하나

514,196 projects

5,370,544 releases

10,282,774 files

784,571 users



? How do I package and publish my code for PyPI?

For full instructions on configuring, packaging and distributing your Python project, refer to the [packaging tutorial](#) on the [Python Packaging User Guide](#).



Python Packaging User Guide

🔍 Search

Overview of Python Packaging

The Packaging Flow

Tutorials

Installing Packages

Managing Application
Dependencies

Packaging Python Projects

Guides

Discussions

PyPA specifications

Project Summaries

Glossary

How to Get Support

Contribute to this guide

News

```
# No user data
ethicalads:
  topic: devs
  region: global
  type: image
```

Packaging Python Projects

This tutorial walks you through how to package a simple Python project. It will show you how to add the necessary files and structure to create the package, how to build the package, and how to upload it to the Python Package Index (PyPI).

Tip

If you have trouble running the commands in this tutorial, please copy the command and its output, then [open an issue](#) on the [packaging-problems](#) repository on GitHub. We'll do our best to help you!

Some of the commands require a newer version of [pip](#), so start by making sure you have the latest version installed:

Unix/macOS

Windows

```
python3 -m pip install --upgrade pip
```

A simple project

This tutorial uses a simple project named `example_package_YOUR_USERNAME_HERE`. If your username is `me`, then the package would be `example_package_me`; this ensures that you have a unique package name that doesn't conflict with packages uploaded by other people following this tutorial. We recommend following this tutorial as-is using this project, before packaging your own project.

Create the following file structure locally:

```
packaging_tutorial/
├── src/
│   ├── example_package_YOUR_USERNAME_HERE/
│   │   ├── __init__.py
│   │   └── example.py
```

ON THIS PAGE

A simple project

Creating the package files

Creating a test directory

Choosing a build backend

Configuring metadata

Creating README.md

Creating a LICENSE

Including other files

Generating distribution archives

Uploading the distribution archives

Installing your newly uploaded
package

Next steps

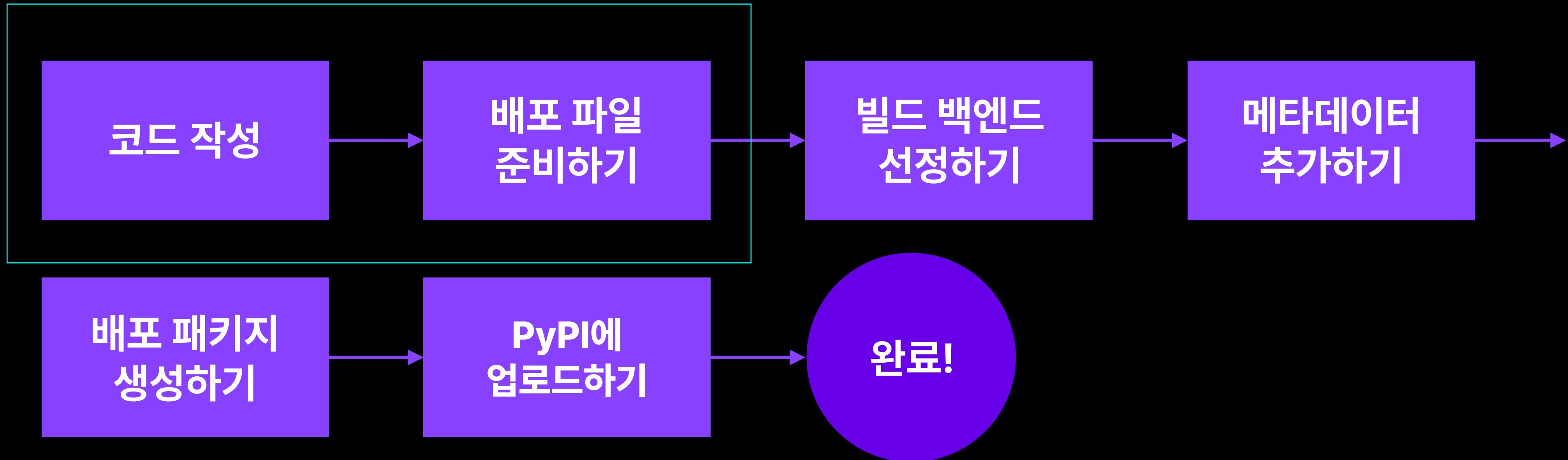


패키지 등록 과정

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기





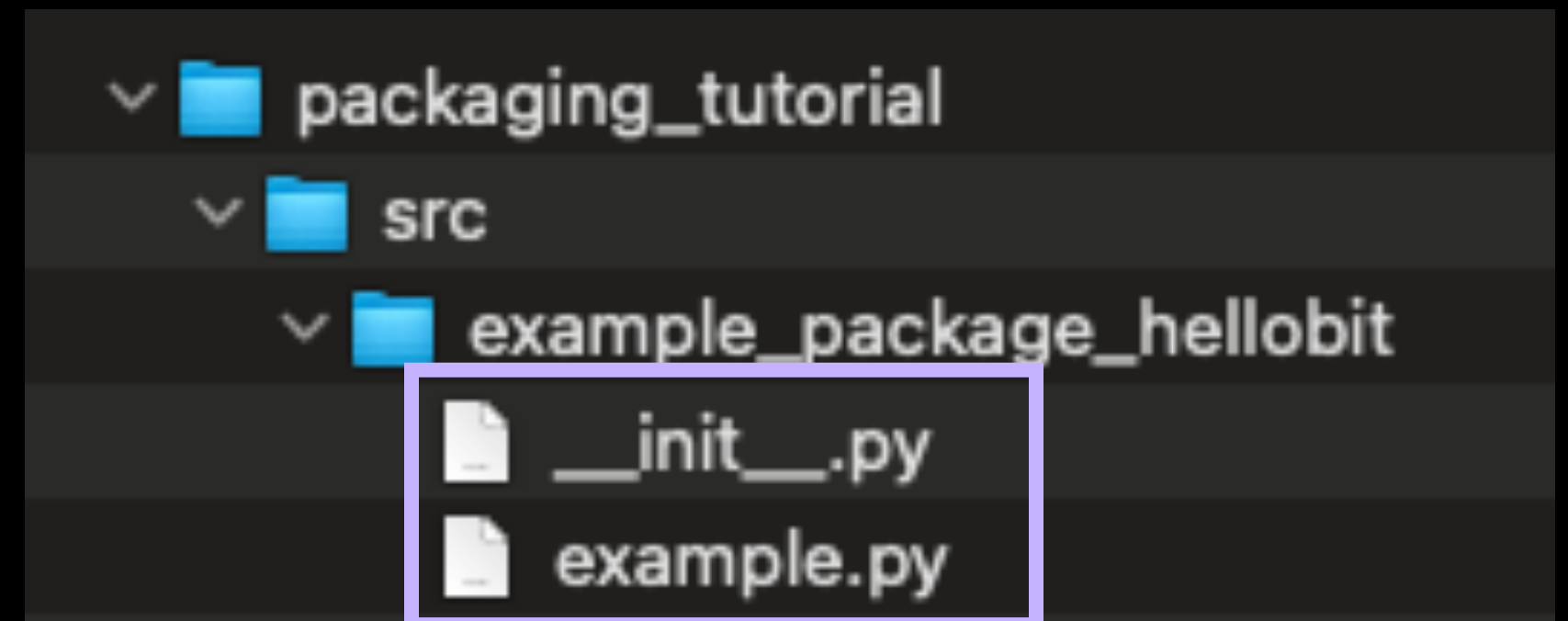
1. 코드 작성

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
packaging_tutorial/  
├── src/  
│   ├── example_package_YOUR_USERNAME_HERE/  
│   │   ├── __init__.py  
│   │   └── example.py
```



- **`__init__.py`** : 디렉터리를 단순한 폴더가 아닌 '패키지'로 인식하도록 함 (권장)
 - 비어있을 수도, 혹은 패키지 초기화 코드를 포함할 수도 있음
- **`example.py`** : 패키지의 주요 로직(함수, 클래스, 상수) 을 포함하는 모듈



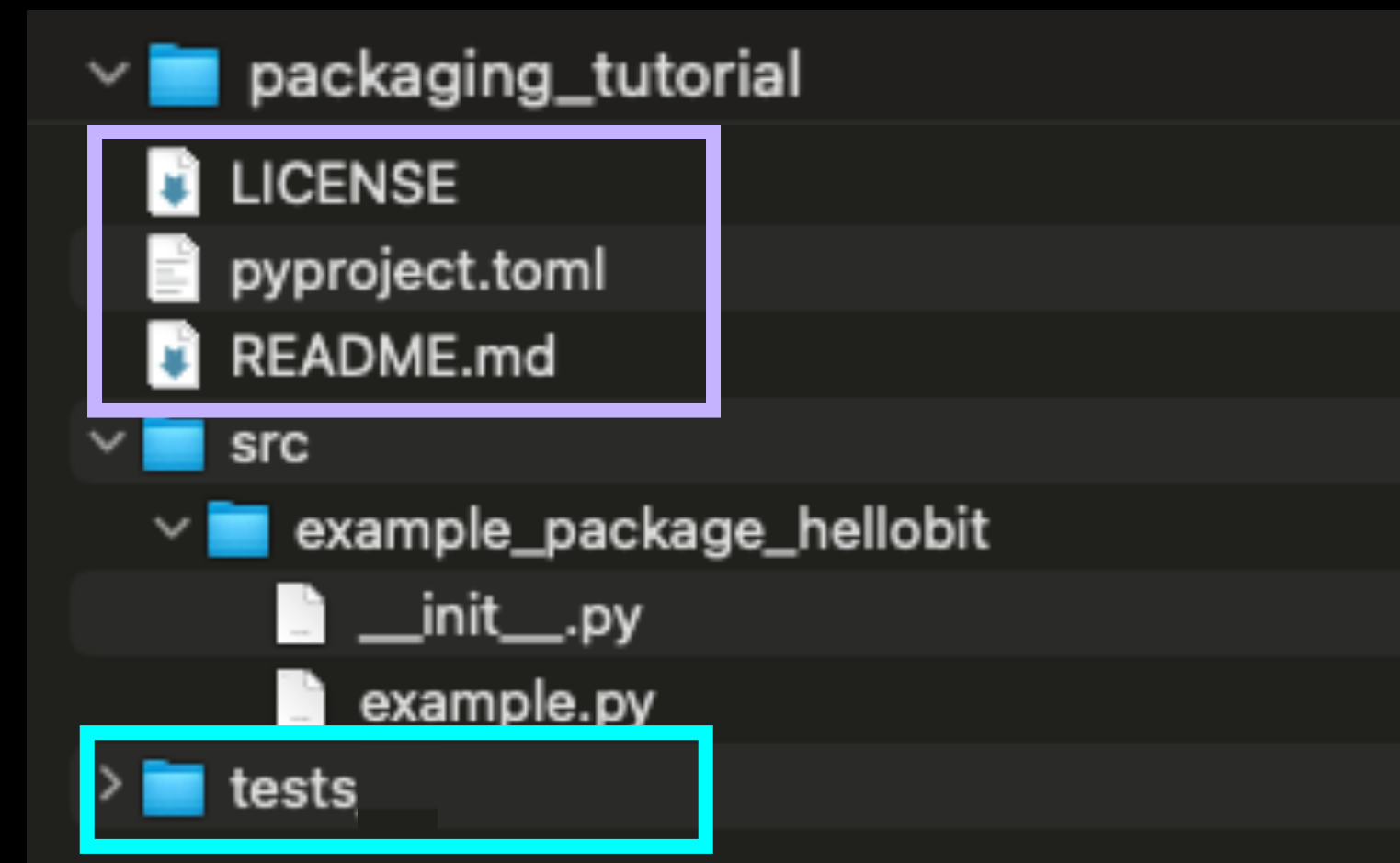
2. 배포 파일 준비하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
packaging_tutorial/  
├── LICENSE  
├── pyproject.toml  
├── README.md  
├── src/  
│   └── example_package_YOUR_USERNAME_HERE/  
│       ├── __init__.py  
│       └── example.py  
└── tests/
```



- **LICENSE** : 패키지를 설치하는 사용자에게 해당 패키지를 사용할 수 있는 조건 명시
- **pyproject.toml** : 빌드 시스템을 명시적으로 정의하고, 패키지 메타데이터 관리 기능 제공
- **README.md** : 프로젝트나 소프트웨어 패키지에 대한 개요, 설치 및 사용 방법 등을 포함하는 마크다운 문서
- **tests** : 테스트 파일을 위한 플레이스홀더 (=임시로 대신하여 자리를 차지하는 기호 / 문자)



- 이번 튜토리얼에서는 별도의 테스트 파일을 생성하지 않으나, 일반적으로는 해당 패키지의 기능이 정상적으로 작동하는지 확인하기 위한 테스트 코드를 작성
- 테스트는 개발 과정에서 발생할 수 있는 오류를 조기에 발견하기 위한 역할

단위 테스트 (Unit Test)

- 작은 단위의 테스트
- 일반적으로 클래스 또는 메서드 수준

통합 테스트 (Integration Test)

- 보다 큰 단위의 테스트
- 일반적으로 여러 모듈들을 모아 확인

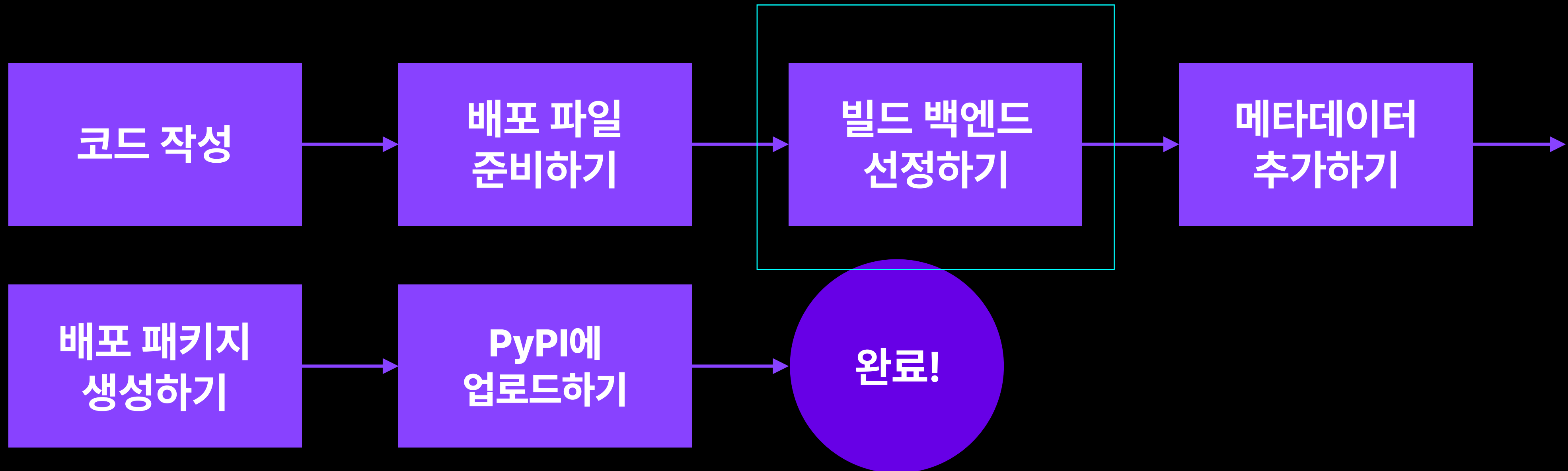


패키지 등록 과정

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기





3. 빌드 백엔드 선정하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

- 개발자가 작성한 소스 코드를 실행 가능한 형태나 배포 가능한 패키지로 변환하는 역할을 수행
- 코드 컴파일, 패키지 빌드, 의존성 관리, 메타데이터 생성 등의 역할 수행
- 파이썬엔 setuptools, flit, poetry, hatchling 등이 있음
- pyproject.toml에 아래와 같이 어떤 빌드 백엔드를 사용하는지 명시

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```




3. 빌드 백엔드 선정하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

- **requires** : 패키지를 빌드하는데 필요한 패키지 목록, 빌드할 때 자동으로 설치
- **build-backend** : 빌드를 수행하기 위해 사용할 파이썬 객체의 이름



3. 빌드 백엔드 선정하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

setuptools

- 파이썬에서 가장 오래되고 널리 사용되는 패키지 배포 도구
- 장점 : 복잡한 프로젝트 지원, 많은 자료와 커뮤니티
- 단점 : 최신 패키지 요구사항을 빠르게 반영하기 어려움

복잡한 프로젝트

단순한 순수 파이썬 프로젝트

flit

- 순수 파이썬 코드만으로 구성된 패키지를 쉽게 배포할 수 있도록 설계
- 장점 : 설정이 간단하고 사용이 매우 쉬움
- 단점 : setuptools보다는 기능이 제한적, 순수 파이썬 코드로만 구성된 패키지에 한정됨

poetry

- 의존성 관리 및 패키지 배포 모두 처리 가능 및 프로젝트의 전체 라이프사이클 관리 가능
- 장점 : 의존성 관리와 패키지 빌드, 배포 모두 처리 가능
- 단점 : 비교적 최근에 개발되어 호환성 문제 발생 가능

의존성 관리를 하나로 처리

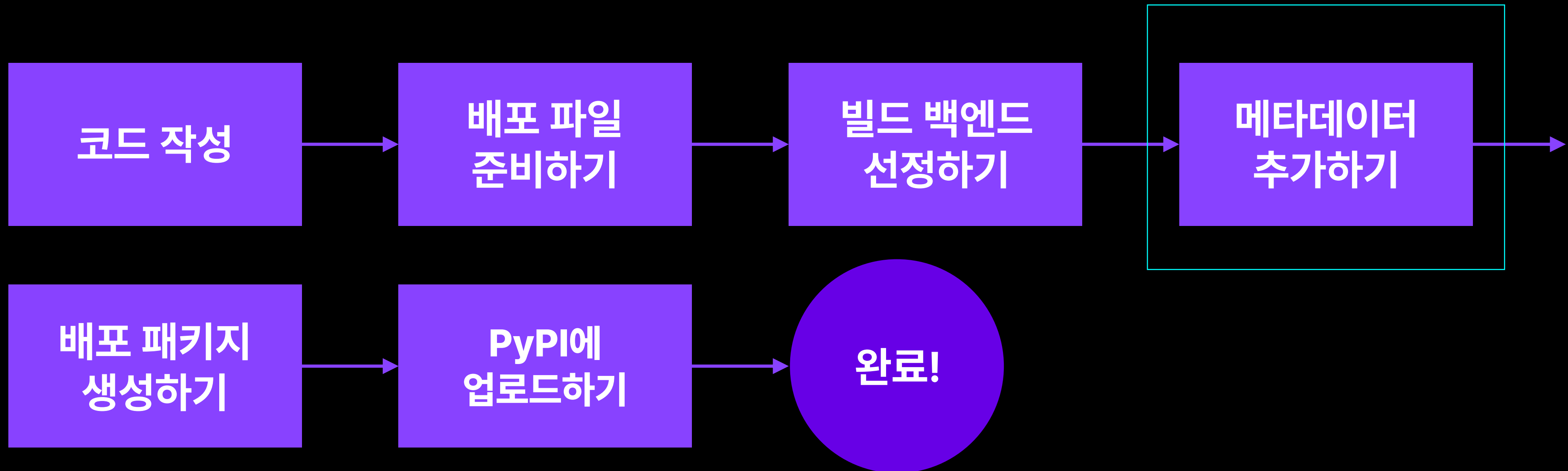


패키지 등록 과정

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기





4. 메타데이터 추가하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
[project]
name = "example_package_YOUR_USERNAME_HERE"
version = "0.0.1"
authors = [
    { name="Example Author", email="author@example.com" },
]
description = "A small example package"
readme = "README.md"
requires-python = ">=3.8"
classifiers = [
    "Programming Language :: Python :: 3",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
]

[project.urls]
Homepage = "https://github.com/pypa/sampleproject"
Issues = "https://github.com/pypa/sampleproject/issues"
```




4. 메타데이터 추가하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
[project]
name = "example_package_YOUR_USERNAME_HERE"
version = "0.0.1"
authors = [
    { name="Example Author", email="author@example.com" },
]
description = "A small example package"
```

- **name** : 패키지의 배포 이름, 문자, 숫자, ., -, _ 를 포함하는 한 어떤 이름이든 가능
 - 다만, 이미 PyPI에서 사용 중인 이름은 불가
- **version** : 패키지 버전
- **authors** : 패키지 개발자의 이름과 이메일 지정
- **description** : 패키지에 대한 짧은 한 문장 소개



4. 메타데이터 추가하기

실전 파이썬 라이브러리
활용

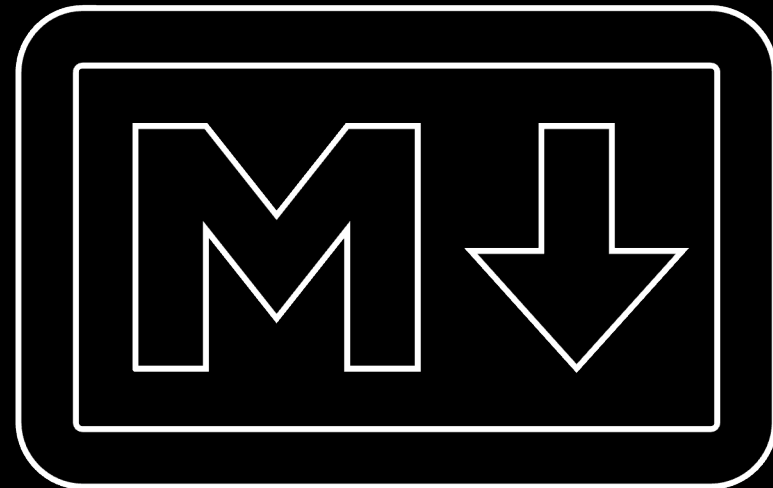
04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
readme = "README.md"
requires-python = ">=3.8"
classifiers = [
    "Programming Language :: Python :: 3",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
]

[project.urls]
Homepage = "https://github.com/pypa/sampleproject"
Issues = "https://github.com/pypa/sampleproject/issues"
```

- **readme** : 패키지에 대한 설명이 포함된 파일 (README.md)의 경로
- **requires-python** : 지원하는 파이썬 버전
- **classifiers** : 패키지에 대한 추가 메타데이터를 인덱스와 pip에 제공
- **urls** : PyPI에 표시할 링크 나열 (문서, 이슈 트래커 등)



- 텍스트를 서식을 적용하여 표현하는데 사용됨
- GitHub와 같은 플랫폼에서 많이 사용되는 개발자가 사랑하는 문서
- 변환 도구로 HTML, PDF, 이미지 파일 등으로 변환할 수 있음

대표적인 마크다운 문법

#	헤더 (#의 개수가 많아질수록 하위 제목)
[]()	대괄호로 텍스트를 감싸고, 바로 뒤에 소괄호로 URL 작성
``	인라인 코드는 백틱(`) 으로 감싸고, 코드 블록은 백틱을 3번 사용(```)



아래와 같이 여러 마크다운 문법을 활용하여 작성

```
# Example Package
```

```
This is a simple example package. You can use  
[GitHub-flavored Markdown](https://guides.github.com/features/mastering-markdown/)  
to write your content.
```

Example Package

This is a simple example package. You can use [GitHub-flavored Markdown](#) to write your content.



1. 라이선스 선택 - <https://choosealicense.com/>

2. 선택한 라이선스에 따라 라이선스 텍스트 작성

Copyright (c) 2018 The Python Packaging Authority

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.



대표적인 라이선스

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

MIT
라이선스

GNU 일반
공중 사용
허가서 (GPL)

Apache
라이선스 2.0

BSD
라이선스

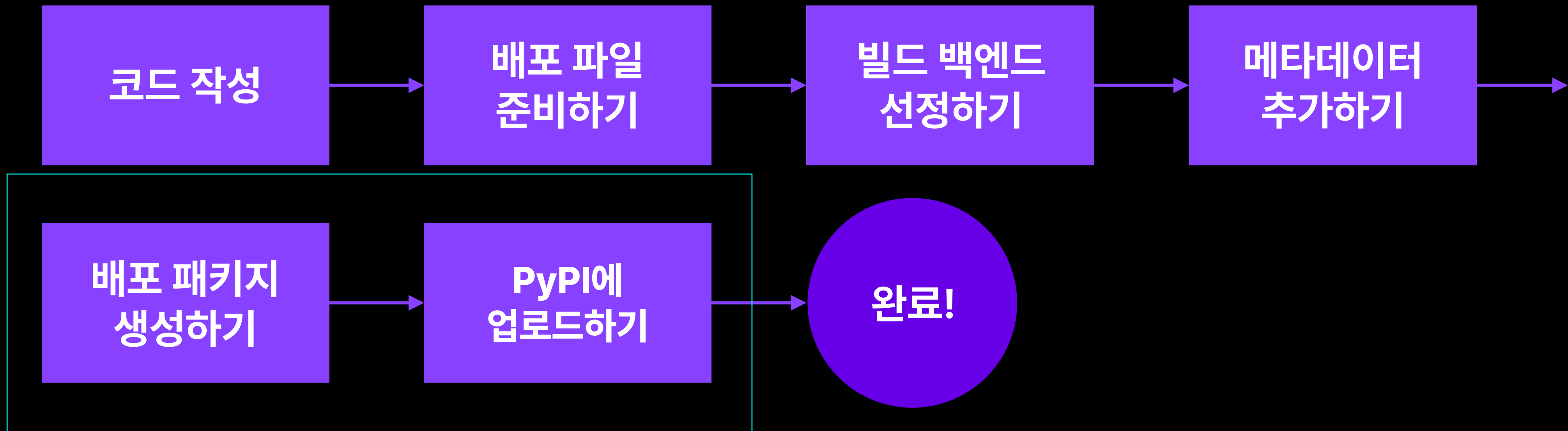


패키지 등록 과정

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기





5. 배포 패키지 생성하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

- wheel 파일과 .tar.gz 생성 단계
- build 라이브러리 설치 필요
- pyproject.toml 파일이 있는 디렉토리에서 아래 명령어 실행

Windows

```
py -m build
```

MacOS/Unix

```
python3 -m build
```




ls / dir	현재 디렉토리의 파일 및 하위 디렉토리 목록 확인
cd 디렉토리	다른 디렉토리로 이동
pwd / cd	현재 작업 중인 디렉토리 경로 출력

▼ packaging_tutorial

LICENSE

pyproject.toml

README.md

▼ src

▼ example_package_hellobit

__init__.py

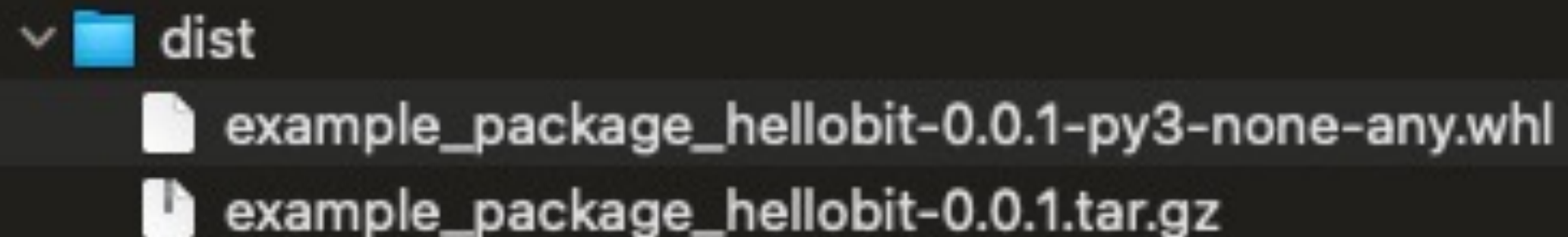
example.py

> tests/

```
cd packaging_tutorial
```



```
dist/
├── example_package_YOUR_USERNAME_HERE-0.0.1-py3-none-any.whl
└── example_package_YOUR_USERNAME_HERE-0.0.1.tar.gz
```



```
dist
├── example_package_hellobit-0.0.1-py3-none-any.whl
└── example_package_hellobit-0.0.1.tar.gz
```

build 라이브러리가 자동적으로 2개의 배포 패키지(whl, tar.gz)를 생성한 것을 확인



6. PyPI에 업로드하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

1. 계정 생성하기

- TestPyPI 계정 등록하기 (TestPyPI : 실제 PyPI에 업로드하지 않고, 테스트하는 목적)
- <https://test.pypi.org/account/register/>
- 실제 PyPI 업로드 방식과 유사
- 인증 이메일 확인 권장

Email address	Status
	PRIMARY ✓ VERIFIED

2. API 토큰 발급

- <https://test.pypi.org/manage/account/#api-tokens> 에서 API 토큰 발급
- "Scope"를 "Entire account"로 설정, Two factor Authentication (2FA) 세팅 필요
- 주의 : 토큰을 반드시 복사해두어야 함!

<https://packaging.python.org/en/latest/tutorials/packaging-projects>



- 계정을 보호하기 위한 다양한 보안 수단
- API 토큰은 사용자 인증 방식의 하나로, 사용자의 비밀번호를 직접 사용하는 것보다 더 안전하고 관리하기 쉬운 방법을 제공
- 2FA란 계정 보호를 위하여 사용자 이름, 비밀번호와 같은 일반적인 인증 수단 외에 추가적인 2번째 인증 수단을 요구하는 보호 절차





6. PyPI에 업로드하기

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

3. pip를 활용하여 twine 라이브러리 설치

```
pip install twine
```

4. 아래 명령어를 사용하여 배포 패키지 업로드

Windows

```
py -m twine upload --repository testpypi dist/*
```

MacOS/Unix

```
python3 -m twine upload --repository testpypi dist/*
```



```
Uploading distributions to https://test.pypi.org/legacy/
Enter your API token:
Uploading example_package_hellobit-0.0.1-py3-none-any.whl
100% ----- 6.2/6.2 kB · 00:00 · ?
Uploading example_package_hellobit-0.0.1.tar.gz
100% ----- 5.7/5.7 kB · 00:00 · ?

View at:
https://test.pypi.org/project/example-package-hellobit/0.0.1/
```

API token을 매번 입력해주고 싶지 않다면, .pypirc 파일을 생성하여 아래와 같이 작성해둬م

```
[pypi]
username = __token__
password = <the token value, including the `pypi-` prefix>
```



TestPyPI에 업로드한 패키지는 아래의 방법으로 설치 가능

Windows

```
py -m pip install --index-url https://test.pypi.org/simple/ --no-deps example-package-YOUR-USERNAME-HERE
```

MacOS/Unix

```
python3 -m pip install --index-url https://test.pypi.org/simple/ --no-deps example-package-YOUR-USERNAME-HERE
```




설치 결과 및 테스트

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기

```
Looking in indexes: https://test.pypi.org/simple/  
Collecting example-package-hellobit  
  Downloading https://test-files.pythonhosted.org/packages/ca/05/2f3a3bdd9f59bcd9b3e8f6d6278be4bb4e5201fd4bf2fb18f2337c645288/example_package_hellobit-0.0.1-py3-none-any.whl (2.7 kB)  
Installing collected packages: example-package-hellobit  
Successfully installed example-package-hellobit-0.0.1
```

```
from example_package_hellobit import example  
  
print(example.add_one(2)) # 3
```

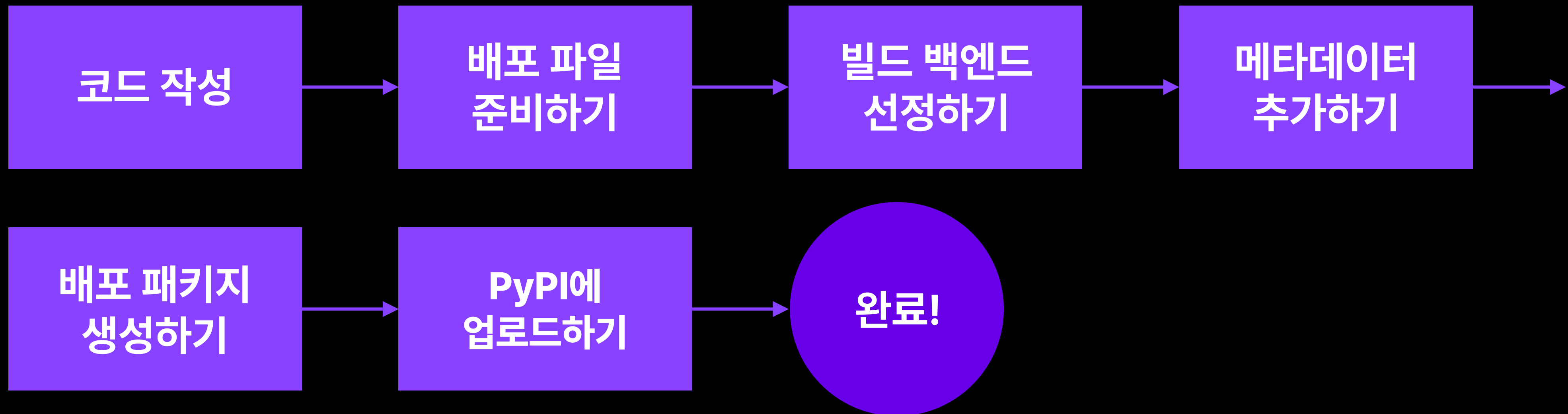



패키지 등록 과정 리뷰

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

2. PyPI에 내 패키지 등
록하기



3. 학습 내용 정리 및 앞으로



open source

- 소스 코드를 공개하여 누구나 자유롭게 사용, 수정, 배포할 수 있도록 허용하는 소프트웨어 개발 및 배포 방식
- 사용의 자유, 연구의 자유, 배포의 자유, 개선의 자유
- 리눅스, 파이썬, 깃 등이 대표적인 오픈 소스 프로젝트
- 기술의 발전에 있어 오픈 소스의 역할은 매우 중요
 - 소프트웨어 개발의 방식을 근본적으로 변화시킴
 - 혁신을 가속화하고, 기술의 대중화 촉진

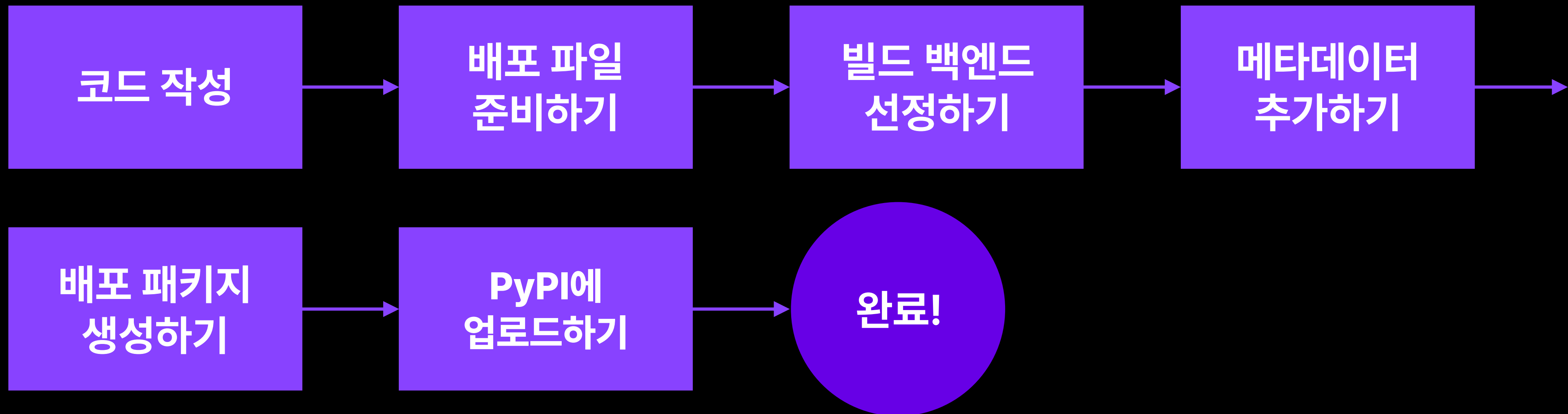


패키지 등록 과정

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

3. 학습 내용 정리 및 앞으
로



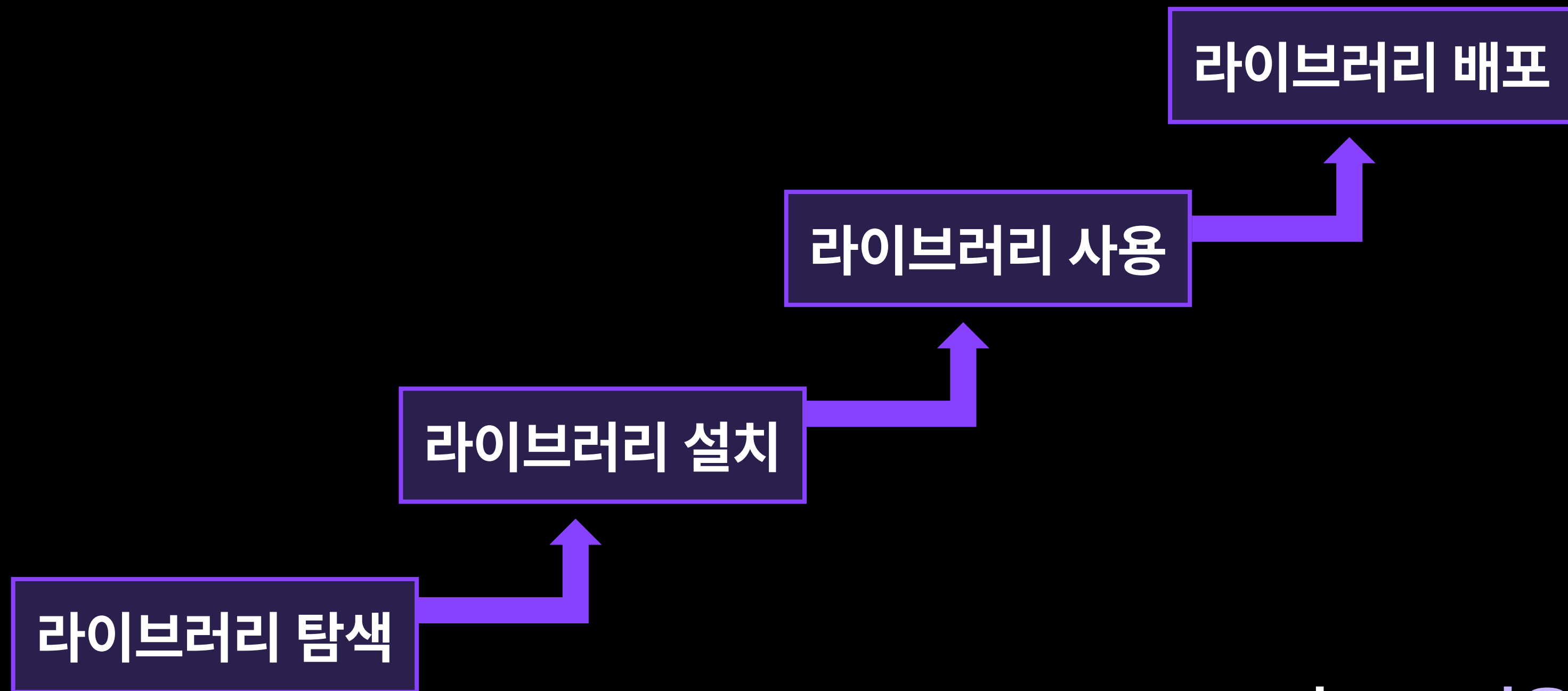


우리는 지금까지...

실전 파이썬 라이브러리
활용

04 내 코드를 전세계 사
람과 공유하기

3. 학습 내용 정리 및 앞으
로



그리고 그 다음은?